
● SAVOL 6:

SETSTYLESHEET() METODI YORDAMIDA GUI ELEMENTLARINI BEZATISH. CSS SINTAKSISI BILAN O'XSHASHLIK VA FARQLAR. MURAKKAB STILLARNI MODULLASHTIRISH USULLARI.

1. setStyleSheet() nima?

setStyleSheet() — PyQt5 da widgetlarni CSS ga o'xshash sintaksis bilan bezash usuli. Qt bu tizimni **QSS (Qt Style Sheets)** deb ataydi.

CSS (web uchun) → brauzerda ishlaydi
QSS (Qt uchun) → PyQt5 da ishlaydi

O'xshashlik: 80% bir xil sintaksis
Farq: 20% Qt ga xos xususiyatlar

2. CSS va QSS — O'xshashliklar

```
/* CSS (web) */
QPushButton {
    background-color: #3498db;
    color: white;
    font-size: 14px;
    font-weight: bold;
    border-radius: 8px;
    padding: 8px 16px;
    border: 2px solid #2980b9;
}
# QSS (PyQt5) – deyarli bir xil!
tugma.setStyleSheet("""
    QPushButton {
        background-color: #3498db;
        color: white;
        font-size: 14px;
        font-weight: bold;
        border-radius: 8px;
        padding: 8px 16px;
        border: 2px solid #2980b9;
    }
""")
```

Ikkisida ham ishlaydigan xususiyatlar:

- ✓ color
- ✓ background-color
- ✓ font-size
- ✓ font-weight
- ✓ font-family

- ✓ font-style (italic)
 - ✓ border
 - ✓ border-radius
 - ✓ padding
 - ✓ margin
 - ✓ width / height
 - ✓ min-width / max-width
 - ✓ min-height / max-height
 - ✓ text-align
 - ✓ text-decoration
 - ✓ opacity
-

3. CSS va QSS — Farqlar

✗ QSS da ISHLAMAYDIGAN CSS xususiyatlari:

```
/* Bular QSS da ishlamaydi! */
display: flex;           /* ✗ yo'q */
position: absolute;      /* ✗ yo'q */
z-index: 10;             /* ✗ yo'q */
box-shadow: ...;         /* ✗ yo'q */
transition: ...;         /* ✗ yo'q — animatsiya yo'q */
transform: rotate();      /* ✗ yo'q */
grid-template: ...;      /* ✗ yo'q */
overflow: hidden;        /* ✗ yo'q */
```

✓ QSS da FAQAT O'ZI bilgan xususiyatlar:

1. Widget nomi selektor sifatida

```
"QPushButton { ... }"    # faqat tugmalar
"QLabel { ... }"         # faqat labellar
"QLineEdit { ... }"      # faqat inputlar
```

2. Pseudo-state lar

```
"QPushButton:hover { background: #2980b9; }" # ustiga kelganda
"QPushButton:pressed { background: #1a6fa8; }" # bosilganda
"QPushButton:disabled { background: #bdc3c7; }" # o'chirilganda
"QPushButton:checked { background: #27ae60; }" # belgilanganda
"QLineEdit:focus { border: 2px solid blue; }" # fokusda
"QLineEdit:read-only { background: #ecf0f1; }" # faqat o'qish
```

3. ID selektor (#objectName)

```
"QPushButton#submitBtn { background: green; }"
```

4. Class selektor (property orqali)

```
"QPushButton[type='primary'] { background: blue; }"
```

5. Subcontrol (widget ichidagi qismlar)

```
"QComboBox::drop-down { width: 30px; }"
"QScrollBar::handle { background: #888; }"
"QCheckBox::indicator { width: 18px; height: 18px; }"
"QProgressBar::chunk { background: #3498db; }"
```

4. Selektor Turlari

1. Universal – barcha widgetlarga

```
app.setStyleSheet("* { font-family: Arial; })
```

2. Widget turi

```
"QPushButton { ... }"
```

3. ID (objectName)

```
tugma.setObjectName("submitBtn")
```

```
"QPushButton#submitBtn { background: green; }"
```

4. Irsiyat (farzand widgetlar)

```
"QWidget QLabel { color: gray; }"    # QWidget ichidagi QLabel lar
```

5. To'g'ridan-to'g'ri farzand

```
"QWidget > QPushButton { ... }"
```

6. Property (xususiyat)

Avval xususiyat o'rnatiladi:

```
tugma.setProperty("role", "danger")
```

Keyin selektor:

```
"QPushButton[role='danger'] { background: red; }"
```

5. Pseudo-State lar To'liq Ro'yxati

```
stili = ""
```

```
/* Hover – sichqoncha ustida */
```

```
QPushButton:hover {  
    background-color: #2980b9;  
    cursor: pointer;  
}
```

```
/* Pressed – bosilganda */
```

```
QPushButton:pressed {  
    background-color: #1a6fa8;  
    padding-top: 2px;    /* bosganday taassurot */  
}
```

```
/* Disabled – o'chirilgan holat */
```

```
QPushButton:disabled {  
    background-color: #bdc3c7;  
    color: #7f8c8d;  
}
```

```
/* Checked – belgilangan (checkable tugma) */
```

```
QPushButton:checked {  
    background-color: #27ae60;  
    border: 2px solid #1e8449;  
}
```

```
/* Focus – klaviatura fokusi */
```

```
QLineEdit:focus {  
    border: 2px solid #3498db;  
    background-color: #eaf4fb;  
}
```

```

    }

    /* Read-only */
    QLineEdit:read-only {
        background-color: #ecf0f1;
        color: #7f8c8d;
    }

    /* Placeholder matni rangi */
    QLineEdit::placeholder {
        color: #bdc3c7;
        font-style: italic;
    }
}
"""

```

6. Murakkab Stillarni Modullashtirish

Usul 1 — O'zgaruvchilar bilan:

```

# ranglar.py
RANGLAR = {
    "asosiy":      "#3498db",
    "muvaaffaq":   "#27ae60",
    "xato":         "#e74c3c",
    "ogohlant":    "#f39c12",
    "matn":         "#2c3e50",
    "fon":          "#f5f6fa",
    "chegara":     "#dfe6e9",
}

# stillar.py
def tugma_stili(rang: str, radius: int = 8) -> str:
    return f"""
        QPushButton {{
            background-color: {rang};
            color: white;
            border-radius: {radius}px;
            padding: 8px 16px;
            font-size: 14px;
            font-weight: bold;
            border: none;
        }}
        QPushButton:hover {{
            background-color: {rang}cc;
        }}
        QPushButton:pressed {{
            background-color: {rang}99;
        }}
    """

# ishlatish
from ranglar import RANGLAR
from stillar import tugma_stili

tugma1.setStyleSheet(tugma_stili(RANGLAR["asosiy"]))

```

```
tugma2.setStyleSheet(tugma_stili(RANGLAR["muvaqqat"]))
tugma3.setStyleSheet(tugma_stili(RANGLAR["xato"]))
```

Usul 2 — Alohida .qss fayl:

```
/* stil.qss fayli */
```

```
QMainWindow {
    background-color: #f5f6fa;
}
```

```
QPushButton {
    background-color: #3498db;
    color: white;
    border-radius: 8px;
    padding: 8px 16px;
    font-size: 14px;
    border: none;
    min-width: 80px;
}
```

```
QPushButton:hover { background-color: #2980b9; }
QPushButton:pressed { background-color: #1a6fa8; }
QPushButton:disabled { background-color: #bdc3c7; color: #7f8c8d; }
```

```
QLineEdit {
    border: 2px solid #dfe6e9;
    border-radius: 6px;
    padding: 8px 12px;
    font-size: 13px;
    background-color: white;
}
```

```
QLineEdit:focus { border-color: #3498db; }
```

```
# Python da o'qib yuklash
```

```
def stil_yuklash(fayl_nomi: str) -> str:
    try:
        with open(fayl_nomi, 'r', encoding='utf-8') as f:
            return f.read()
    except FileNotFoundError:
        print(f"Stil fayli topilmadi: {fayl_nomi}")
        return ""
```

```
# Ishlatish
```

```
app = QApplication(sys.argv)
app.setStyleSheet(stil_yuklash("stil.qss"))
```

Usul 3 — Tema klassi:

```
class Tema:
    """Markazlashgan stil boshqaruvi"""
```

```
SVETLIY = {
    "fon": "#ffffff",
    "fon2": "#f5f6fa",
    "matn": "#2c3e50",
```

```

        "chegara":      "#dfe6e9",
        "asosiy":       "#3498db",
        "muvaaffaq":    "#27ae60",
        "xato":         "#e74c3c",
    }

    QORONG = {
        "fon":          "#1e272e",
        "fon2":         "#2c3e50",
        "matn":         "#ecf0f1",
        "chegara":      "#4a5568",
        "asosiy":       "#3498db",
        "muvaaffaq":    "#27ae60",
        "xato":         "#e74c3c",
    }

    @staticmethod
    def qo_llash(app: QApplication, tema: dict):
        app.setStyleSheet(f"""
            QMainWindow, QWidget {{
                background-color: {tema['fon']};
                color: {tema['matn']};
            }}
            QPushButton {{
                background-color: {tema['asosiy']};
                color: white;
                border-radius: 8px;
                padding: 8px 16px;
                border: none;
                font-size: 13px;
            }}
            QPushButton:hover {{
                background-color: {tema['asosiy']}cc;
            }}
            QLineEdit {{
                background-color: {tema['fon2']};
                color: {tema['matn']};
                border: 2px solid {tema['chegara']};
                border-radius: 6px;
                padding: 8px;
            }}
            QLineEdit:focus {{
                border-color: {tema['asosiy']};
            }}
        """)

```

Ishlatish:

```

Tema.qo_llash(app, Tema.SVETLIY)    # Svetliy tema
Tema.qo_llash(app, Tema.QORONG)     # Qorong' tema

```

Usul 4 — Komponent funksiyalari:

```

class Stillar:
    """Har bir komponent uchun alohida stil funksiyasi"""

    @staticmethod

```

```

def karta() -> str:
    return """
        QWidget {
            background-color: white;
            border-radius: 12px;
            border: 1px solid #dfe6e9;
        }
    """

@staticmethod
def asosiy_tugma() -> str:
    return """
        QPushButton {
            background-color: #3498db;
            color: white;
            border-radius: 8px;
            padding: 10px 24px;
            font-size: 14px;
            font-weight: bold;
            border: none;
        }
        QPushButton:hover { background-color: #2980b9; }
        QPushButton:pressed { background-color: #1a6fa8; }
    """

@staticmethod
def xavfli_tugma() -> str:
    return """
        QPushButton {
            background-color: #e74c3c;
            color: white;
            border-radius: 8px;
            padding: 10px 24px;
            font-size: 14px;
            font-weight: bold;
            border: none;
        }
        QPushButton:hover { background-color: #c0392b; }
        QPushButton:pressed { background-color: #a93226; }
    """

@staticmethod
def input_maydoni() -> str:
    return """
        QLineEdit {
            border: 2px solid #dfe6e9;
            border-radius: 8px;
            padding: 10px 14px;
            font-size: 14px;
            background: white;
            color: #2c3e50;
        }
        QLineEdit:focus { border-color: #3498db; }
        QLineEdit:hover { border-color: #bdc3c7; }
        QLineEdit:disabled { background: #ecf0f1; }
    """

```

```
# Ishlatish:
karta_widget.setStyleSheet(Stillar.karta())
saqlash_tugma.setStyleSheet(Stillar.asosiy_tugma())
ochirish_tugma.setStyleSheet(Stillar.xavfli_tugma())
ism_input.setStyleSheet(Stillar.input_maydoni())
```

7. To'liq Amaliy Misol

```
import sys
from PyQt5.QtWidgets import *
from PyQt5.QtCore import Qt
```

```
# == Tema va stillar ==
ASOSIY_STIL = """
    QMainWindow {
        background-color: #f0f2f5;
    }
    QWidget#markaziy {
        background-color: #f0f2f5;
    }
    QLabel#sarlavha {
        font-size: 22px;
        font-weight: bold;
        color: #2c3e50;
        padding: 10px 0;
    }
    QLabel#kichik {
        font-size: 13px;
        color: #7f8c8d;
    }
    QPushButton#asosiy {
        background-color: #3498db;
        color: white;
        border-radius: 10px;
        padding: 12px 28px;
        font-size: 15px;
        font-weight: bold;
        border: none;
        min-width: 120px;
    }
    QPushButton#asosiy:hover { background-color: #2980b9; }
    QPushButton#asosiy:pressed { background-color: #1a6fa8; }
    QPushButton#xavfli {
        background-color: #e74c3c;
        color: white;
        border-radius: 10px;
        padding: 12px 28px;
        font-size: 15px;
        font-weight: bold;
        border: none;
    }
    QPushButton#xavfli:hover { background-color: #c0392b; }
    QPushButton#xavfli:pressed { background-color: #a93226; }
    QPushButton#yashil {
```



```

        background-color: #27ae60;
        color: white;
        border-radius: 10px;
        padding: 12px 28px;
        font-size: 15px;
        font-weight: bold;
        border: none;
    }
QPushButton#yashil:hover { background-color: #219a52; }
QPushButton#yashil:pressed { background-color: #1a7a40; }
QLineEdit#input {
    border: 2px solid #dfe6e9;
    border-radius: 10px;
    padding: 10px 16px;
    font-size: 14px;
    background: white;
    color: #2c3e50;
    min-height: 20px;
}
QLineEdit#input:focus { border-color: #3498db; background: #eaf4fb; }
QLineEdit#input:hover { border-color: #aab7b8; }
QProgressBar {
    border: none;
    border-radius: 8px;
    background-color: #dfe6e9;
    height: 16px;
    text-align: center;
    font-size: 11px;
    color: white;
}
QProgressBar::chunk {
    border-radius: 8px;
    background-color: #3498db;
}
QCheckBox {
    font-size: 14px;
    color: #2c3e50;
    spacing: 8px;
}
QCheckBox::indicator {
    width: 20px;
    height: 20px;
    border-radius: 4px;
    border: 2px solid #bdc3c7;
    background: white;
}
QCheckBox::indicator:checked {
    background-color: #3498db;
    border-color: #3498db;
}
"""

```

```

class StilDemoDastur(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("setStyleSheet Demo")

```

```

self.setFixedSize(480, 560)
self.setStyleSheet(ASOSIY_STIL)
self.interfeys_yaratish()

def interfeys_yaratish(self):
    markaziy = QWidget()
    markaziy.setObjectName("markaziy")
    self.setCentralWidget(markaziy)
    tartib = QVBoxLayout(markaziy)
    tartib.setContentsMargins(40, 30, 40, 30)
    tartib.setSpacing(16)

    # Sarlavha
    sarlavha = QLabel("setStyleSheet Demo")
    sarlavha.setObjectName("sarlavha")
    sarlavha.setAlignment(Qt.AlignCenter)
    tartib.addWidget(sarlavha)

    # Input maydonlari
    self.ism = QLineEdit()
    self.ism.setObjectName("input")
    self.ism.setPlaceholderText("Ismingiz...")
    tartib.addWidget(self.ism)

    self.email = QLineEdit()
    self.email.setObjectName("input")
    self.email.setPlaceholderText("Email...")
    tartib.addWidget(self.email)

    # Checkbox
    self.eslab = QCheckBox("Meni eslab qol")
    tartib.addWidget(self.eslab)

    # Progress bar
    self.progress = QProgressBar()
    self.progress.setValue(65)
    tartib.addWidget(self.progress)

    # Tugmalar
    tugmalar = QHBoxLayout()
    tugmalar.setSpacing(12)

    btn1 = QPushButton("Kirish")
    btn1.setObjectName("asosiy")

    btn2 = QPushButton("O'chirish")
    btn2.setObjectName("xavfli")

    btn3 = QPushButton("Saqlash")
    btn3.setObjectName("yashil")

    tugmalar.addWidget(btn1)
    tugmalar.addWidget(btn2)
    tugmalar.addWidget(btn3)
    tartib.addLayout(tugmalar)

    # Kichik izoh

```

```

izoh = QLabel("Barcha stillar setStyleSheet() orqali berilgan")
izoh.setObjectName("kichik")
izoh.setAlignment(Qt.AlignCenter)
tartib.addWidget(izoh)

```

```

app = QApplication(sys.argv)
oyna = StilDemoDastur()
oyna.show()
sys.exit(app.exec_())

```

8. Xulosa Jadvali

Xususiyat	CSS (web)	QSS (PyQt5)
color , background	✓	✓
border , padding , margin	✓	✓
font-size , font-weight	✓	✓
border-radius	✓	✓
:hover , :focus , :disabled	✓	✓
box-shadow	✓	✗
transition , animation	✓	✗
display: flex	✓	✗
Widget nomi selektor	✗	✓
::subcontrol	✗	✓
.qss fayl	✗	✓

Modullashtirish usullari:

1. O'zgaruvchilar → ranglar dict + f-string
2. .qss fayl → alohida stillar fayli
3. Tema klassi → Svetliy/Qorong' tema
4. Komponent funksiya → har widget uchun alohida

💡 **Eslab qol:** `setObjectName()` + `#id` selektor — eng kuchli usul! Bir xil widget turlarini alohida bezash imkonini beradi.

● SAVOL 7:

QTDESIGNER DASTURINING IMKONIYATLARI: DRAG-AND-DROP DIZAYN, .UI FAYLLARNI PYTHON KODIGA O'TKAZISH (PYUIC5) HAMDA QO'LDA KOD YOZISH BILAN SOLISHTIRGANDA AFZALLIKLARI.

1. QtDesigner nima?

QtDesigner — PyQt5 uchun vizual interfeys yaratish dasturi. Kod yozmasdan, sudrab-tashlab (drag-and-drop) usulida oynalar, formalar va dialoglar yaratish imkonini beradi.

Kod yozish usuli:

QtDesigner usuli:

```
tugma = QPushButton() → [Tugmani sudrab tashlash]
tugma.setText("OK") → [Xususiyatlar panelida yozish]
tartib.addWidget(tugma) → [Avtomatik]
```

2. QtDesigner ni O'rnatish

PyQt5 bilan birga keladi:

```
pip install PyQt5
```

```
pip install pyqt5-tools    # ← Designer shu paketda
```

Ishga tushirish:

Windows:

```
designer.exe
```

yoki:

```
python -m pyqt5tools designer
```

Linux/Mac:

```
designer
```

3. QtDesigner Interfeysi

Widget Box	Dizayn maydoni	Inspector
- Layouts - VBox - HBox - Grid - Buttons - PushButton - CheckBox - RadioBtn - Input - LineEdit - TextEdit - ComboBox - Display - Label - ProgressBar	Sizning oyna [Label] [LineEdit] [OK]	Object Hierarchy Property Editor objectName geometry font text ... Signal/Slot Editor

4. Drag-and-Drop Dizayn Jarayoni

- 1-qadam: Widget Box dan widget tanlang
(masalan: QPushButton)
↓
 - 2-qadam: Dizayn maydoniga sudrab tashlang
↓
 - 3-qadam: Property Editor da xususiyatlarni o'rnatish
 - objectName: "loginBtn"
 - text: "Kirish"
 - geometry: x, y, width, height↓
 - 4-qadam: Layout qo'llang (Ctrl+L = VBox, Ctrl+H = HBox)
↓
 - 5-qadam: Signal/Slot editor da bog'lang
↓
 - 6-qadam: Fayl → Saqlash → login.ui
-

5. .ui Fayl nima?

.ui fayl — QtDesigner tomonidan yaratilgan **XML** formatidagi fayl. Ichida widgetlar, ularning xususiyatlari va joylashuvi yozilgan.

```
<!-- login.ui fayli (XML ko'rinishi) -->
<?xml version="1.0" encoding="UTF-8"?>
<ui version="4.0">
  <class>LoginForm</class>
  <widget class="QWidget" name="LoginForm">
    <property name="geometry">
      <rect><x>0</x><y>0</y><width>400</width><height>300</height></rect>
    </property>
    <layout class="QVBoxLayout" name="verticalLayout">
      <item>
        <widget class="QLabel" name="sarlavha_label">
          <property name="text"><string>Kirish</string></property>
        </widget>
      </item>
      <item>
        <widget class="QLineEdit" name="login_input">
          <property name="placeholderText"><string>Login...</string></property>
        </widget>
      </item>
      <item>
        <widget class="QPushButton" name="kirish_btn">
          <property name="text"><string>Kirish</string></property>
        </widget>
      </item>
    </layout>
  </widget>
</ui>
```

6. pyuic5 — .ui dan .py ga O'tkazish

pyuic5 — `.ui` faylni Python kodiga aylantiradigan buyruq qatori vositasi.

Asosiy ishlatish:

```
pyuic5 login.ui -o ui_login.py
```

Yoki:

```
python -m PyQt5.uic.pyuic login.ui -o ui_login.py
```

Barcha .ui fayllarni bir vaqtda:

```
for file in *.ui; do pyuic5 "$file" -o "ui_${file%.ui}.py"; done
```

Yaratilgan ui_login.py ko'rinishi:

ui_login.py (avtomatik yaratilgan — O'ZGARTIRMANG!)

```
from PyQt5 import QtCore, QtGui, QtWidgets
```

```
class Ui_LoginForm(object):
```

```
    def setupUi(self, LoginForm):
```

```
        LoginForm.setObjectName("LoginForm")
```

```
        LoginForm.resize(400, 300)
```

```
        self.verticalLayout = QtWidgets.QVBoxLayout(LoginForm)
```

```
        self.sarlavha_label = QtWidgets.QLabel(LoginForm)
```

```
        self.sarlavha_label.setText("Kirish")
```

```
        self.verticalLayout.addWidget(self.sarlavha_label)
```

```
        self.login_input = QtWidgets.QLineEdit(LoginForm)
```

```
        self.login_input.setPlaceholderText("Login...")
```

```
        self.verticalLayout.addWidget(self.login_input)
```

```
        self.kirish_btn = QtWidgets.QPushButton(LoginForm)
```

```
        self.kirish_btn.setText("Kirish")
```

```
        self.verticalLayout.addWidget(self.kirish_btn)
```

```
    def retranslateUi(self, LoginForm):
```

```
        _translate = QtCore.QCoreApplication.translate
```

```
        LoginForm.setWindowTitle(_translate("LoginForm", "Login"))
```

7. Yaratilgan UI ni Ishlatish — 2 Usul

Usul 1 — Inheritance (Meros olish) Tavsiya etiladi:

main.py

```
import sys
```

```
from PyQt5.QtWidgets import QApplication, QWidget, QMessageBox
```

```
from ui_login import Ui_LoginForm    # ← avtomatik yaratilgan
```

```
class LoginOyna(QWidget, Ui_LoginForm):
```

```
    def __init__(self):
```

```
        super().__init__()
```

```
        self.setupUi(self)           # ← UI ni yuklash
```

```
    # Signal-Slot bog'lash (bu yerda yozamiz)
```

```

self.kirish_btn.clicked.connect(self.kirish)

def kirish(self):
    login = self.login_input.text()
    if login:
        QMessageBox.information(self, "OK", f"Xush kelibsiz, {login}!")
    else:
        QMessageBox.warning(self, "Xato", "Loginni kiriting!")

app = QApplication(sys.argv)
oyna = LoginOyna()
oyna.show()
sys.exit(app.exec_())

```

Usul 2 — uic.loadUi() (to'g'ridan-to'g'ri yuklash):

```

# main.py
import sys
from PyQt5.QtWidgets import QApplication, QWidget
from PyQt5 import uic # ← to'g'ridan-to'g'ri yuklash

class LoginOyna(QWidget):
    def __init__(self):
        super().__init__()
        uic.loadUi("login.ui", self) # ← .ui faylni bevosita yuklash

        # Widgetlarga to'g'ridan-to'g'ri murojaat
        self.kirish_btn.clicked.connect(self.kirish)

    def kirish(self):
        print(self.login_input.text())

app = QApplication(sys.argv)
oyna = LoginOyna()
oyna.show()
sys.exit(app.exec_())

```

Usul 1 vs Usul 2:

Usul 1 (pyuic5):

- ✓ Tezroq ishlaydi (oldindan kompilyatsiya)
- ✓ Xatolarni oldindan ko'rish mumkin
- ✓ IDE da autocomplete ishlaydi
- ✗ Har o'zgarishda pyuic5 qayta ishlatish kerak

Usul 2 (loadUi):

- ✓ Tez ishlab chiqish — o'zgarish darhol ko'rinadi
- ✓ Faqat .ui fayl kerak
- ✗ Sekinroq (har safar XML parse qiladi)
- ✗ IDE da autocomplete ishlamaydi

8. To'g'ri Arxitektura — UI va Mantiqni Ajratish

 TO'G'RI – UI va mantiq ajratilgan

ui_login.py ← QtDesigner yaratadi (O'ZGARTIRMA)

login.py ← Siz yozasiz (faqat mantiq)

login.py

```
from PyQt5.QtWidgets import QWidget, QMessageBox
```

```
from ui_login import Ui_LoginForm
```

```
class LoginOyna(QWidget, Ui_LoginForm):
```

```
    def __init__(self):
```

```
        super().__init__()
```

```
        self.setupUi(self)
```

```
        self._signallarni_boglash()
```

```
    def _signallarni_boglash(self):
```

```
        self.kirish_btn.clicked.connect(self.kirish)
```

```
        self.parol_input.returnPressed.connect(self.kirish)
```

```
    def kirish(self):
```

```
        """Faqat mantiq – dizayn emas"""
```

```
        login = self.login_input.text().strip()
```

```
        parol = self.parol_input.text()
```

```
        if not login or not parol:
```

```
            QMessageBox.warning(self, "Xato", "Barcha maydonlarni to'ldiring!")
```

```
            return
```

```
        if self._tekshir(login, parol):
```

```
            QMessageBox.information(self, "OK", "Muvaffaqiyatli!")
```

```
        else:
```

```
            QMessageBox.critical(self, "Xato", "Login yoki parol noto'g'ri!")
```

```
    def _tekshir(self, login, parol):
```

```
        return login == "admin" and parol == "1234"
```

9. QtDesigner vs Qo'lda Kod — Taqqoslash

Xususiyat	QtDesigner	Qo'lda kod
Tezlik	 Tez	 Sekin
Vizual ko'rinish	 Darhol ko'rinadi	 Ishga tushirish kerak
Murakkab layoutlar	 Oson	 Qiyin
Dinamik UI	 Qiyin	 Oson
Kod nazorati	 Cheklangan	 To'liq nazorat
Jamoaviy ish	 UI ajratilgan	 Chalkash
O'rganish	 Oson	 Vaqt talab
Kichik loyiha	 Ideal	 Ham yaxshi
Katta loyiha	 Tavsiya	 Qiyin

10. Xulosa

QtDesigner:

- Drag-and-drop bilan UI yaratish
- .ui (XML) fayl saqlaydi

pyuic5:

- .ui → .py ga o'tkazadi
- Avtomatik yaratilgan kodni O'ZGARTIRMA!

Ishlatish:

1. Inheritance usuli → production uchun
2. loadUi() usuli → tez prototiplash uchun

Afzalliklari:

- ✓ UI va mantiq ajratiladi
- ✓ Vizual, tez, qulay
- ✓ Katta loyihalarda tartibli

Kamchiliklari:

- ✗ Dinamik UI uchun qo'lda kod yaxshiroq
- ✗ Har o'zgarishda pyuic5 qayta ishlatish kerak

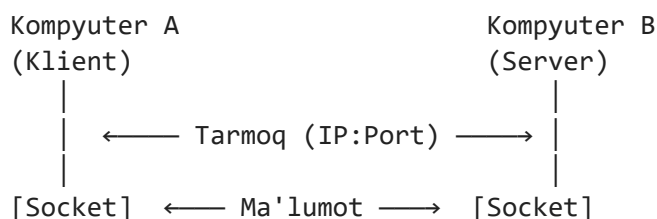
💡 **Eslab qol:** Eng yaxshi yondashuv — **aralash usul:** dizaynni QtDesigner da, mantiqni alohida .py faylda yozish!

● SAVOL 8:

PYTHON SOCKET MODULI ASOSIDA TCP KLIENT-SERVER ARXITEKTURASI. SOCKET(), BIND(), LISTEN(), ACCEPT(), CONNECT(), SEND(), RECV(), CLOSE() METODLARINING KETMA-KETLIGI VA VAZIFALARI.

1. Socket nima?

Socket — bu ikki dastur o'rtasida tarmoq orqali ma'lumot almashish uchun ishlatiladigan **eshik** (endpoint). Xuddi telefon kabi: biri qo'ngiroq qiladi, ikkinchisi qabul qiladi, keyin ikkalasi gaplashadi.



2. TCP nima va nima uchun ishlatiladi?

TCP (Transmission Control Protocol) — ishonchli, tartibli, xatosiz ma'lumot uzatish protokoli.

TCP kafolatlari:

- ✓ Ma'lumot yetib boradi (yo'qolmaydi)
- ✓ Tartib saqlanadi (1,2,3 → 1,2,3)
- ✓ Xatolar tekshiriladi
- ✓ Ulanish o'rnatiladi (握手 – handshake)

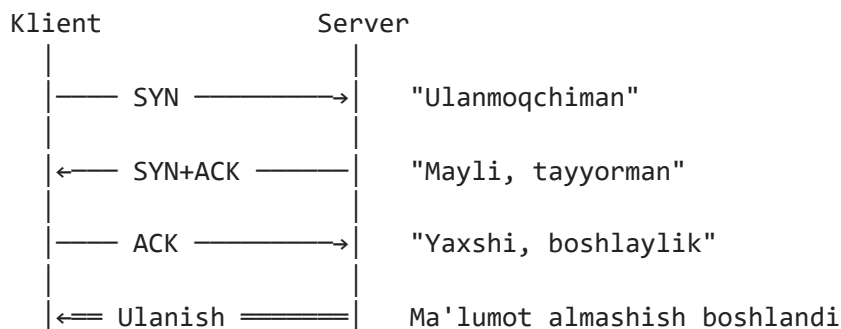
Qachon ishlatiladi:

- Veb saytlar (HTTP)
- Elektron pochta (SMTP)
- Fayl uzatish (FTP)
- Chat dasturlari

TCP vs UDP:

TCP → Sekinroq, lekin ishonchli (xat jo'natish kabi)
UDP → Tezroq, lekin ishonchsiz (radio eshittirish kabi)

3. TCP Ulanish Jarayoni — 3 Bosqichli Handshake



Bu jarayon avtomatik — Python socket kutubxonasi buni o'zi bajaradi.

4. Metodlar Ketma-ketligi

SERVER tomoni:

```
socket()
↓
bind()
↓
listen()
↓
accept() ← Kutadi
↓
send()/recv()
↓
close()
```

KLIENT tomoni:

```
socket()
↓
connect() ← Serverga ulanish
↓
send() ← Ma'lumot yuborish
↓
recv() ← Javob olish
↓
close()
```

5. Har bir Metodning Vazifasi

● socket() — Eshik yaratish

```
import socket
```

```
# AF_INET = IPv4 manzil oilasi
# SOCK_STREAM = TCP (oqim asosida)
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

Nazariy asos: Socket yaratilganda OS xotirada yangi aloqa nuqtasi ajratadi. `AF_INET` — IPv4 manzillar (192.168.x.x), `SOCK_STREAM` — TCP ning belgisi (oqim uzluksiz keladi).

● bind() — Manzil va port biriktirish

```
server.bind(("0.0.0.0", 12345))
# "0.0.0.0" = barcha tarmoq interfeyslarini tinglash
# 12345 = port raqami
```

Nazariy asos: `bind()` serverga "Men shu port orqali kelgan ulanishlarni qabul qilaman" deydi. Port — 0 dan 65535 gacha. 0-1023 = tizim portlari (HTTP=80, HTTPS=443). Bizning dasturlar uchun 1024+ ishlatiladi. `"0.0.0.0"` — barcha tarmoq kartalarini tinglaydi, `"127.0.0.1"` — faqat o'z kompyuterini.

● listen() — Ulanishlarni kutish rejimi

```
server.listen(5)
# 5 = navbat uzunligi (backlog)
```

Nazariy asos: `listen()` socketni **passiv rejimga** o'tkazadi — endi u ulanishlarni kutadi. `5` — bir vaqtda navbatda kutishi mumkin bo'lgan ulanishlar soni. Agar 5 ta ulanish navbatda bo'lsa, 6-chi ulanish rad etiladi.

● accept() — Ulanishni qabul qilish

```
klient_socket, klient_manzil = server.accept()
# accept() – yangi ulanish kelguncha TO'XTAB TURADI (blocking)
# klient_socket = yangi ulanish bilan ishlash uchun socket
# klient_manzil = (IP, port) – klientning manzili
```

Nazariy asos: `accept()` — **blokirovchi** funksiya, ya'ni ulanish kelmaguncha dastur bu qatorda to'xtab turadi. Ulanish kelganda OS yangi socket ob'ektini yaratadi. Asl server socketi esa keyingi ulanishlarni kutishda davom etadi. Bu ikki socket — turli vazifalar uchun!

```
server_socket → Faqat yangi ulanishlarni qabul qiladi
klient_socket → Shu klient bilan gaplashish uchun
```

● connect() — Serverga ulanish (klient tomoni)

```
klient = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
klient.connect(("127.0.0.1", 12345))
```

Nazariy asos: `connect()` TCP handshake ni boshlaydi. Bu yerda OS SYN paketi yuboradi, SYN+ACK kutadi, ACK qaytaradi. Agar server mavjud bo'lmasa — `ConnectionRefusedError` xatosi chiqadi. Agar server band bo'lsa — `TimeoutError`.

● send() / recv() — Ma'lumot yuborish va qabul qilish

Yuborish

```
xabar = "Salom Server!".encode("utf-8") # str → bytes
klient.send(xabar)
```

Qabul qilish

```
data = klient_socket.recv(1024) # max 1024 bayt
matn = data.decode("utf-8") # bytes → str
```

Nazariy asos:

`send()` — ma'lumotni TCP bufferiga qo'yadi, OS uni paketlarga bo'lib yuboradi. **Muhim:** `send()` barcha ma'lumotni bir anda yubormagan bo'lishi mumkin! Katta ma'lumotlar uchun `sendall()` ishlatiladi.

`recv()` — TCP bufferidan ma'lumot oladi. `1024` — bir safarda maksimal necha bayt olinishini bildiradi. Agar ma'lumot 1024 baytdan ko'p bo'lsa — bir necha marta `recv()` chaqirilishi kerak. `recv()` ham blokirovchi — ma'lumot kelgunga qadar kutadi.

To'g'ri katta ma'lumot yuborish:

```
klient.sendall(katta_matn.encode("utf-8")) # ← hammasi yetib borishini kafolatlaydi
```

To'g'ri katta ma'lumot qabul qilish:

```
barcha_data = b""
while True:
    qism = klient_socket.recv(4096)
    if not qism:
        break
    barcha_data += qism
```

● close() — Ulanishni yopish

```
klient_socket.close()
server.close()
```

Nazariy asos: `close()` TCP **FIN** paketi yuboradi — "Men tugatdim" degan signal. Bu TCP ning to'g'ri yopilishi (graceful shutdown). OS socket uchun ajratilgan xotirani bo'shatadi. Yopilgan socketlar — **resurs sizishi (resource leak)** ga olib keladi. `with` operatori bilan socketni avtomatik yopish tavsiya etiladi:

```
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.connect(("127.0.0.1", 12345))
    s.sendall(b"Salom!")
    # with bloki tugaganda s.close() avtomatik chaqiriladi
```

6. Oddiy Server va Klient

=== SERVER ===

```
import socket
```

```
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
# SO_REUSEADDR → dastur qayta ishga tushganda portni darhol ishlatish
```

```

server.bind(("0.0.0.0", 12345))
server.listen(5)
print("Server kutmoqda...")

klient_sock, manzil = server.accept()
print(f"Ulandi: {manzil}")

xabar = klient_sock.recv(1024).decode("utf-8")
print(f"Keldi: {xabar}")

klient_sock.send("Xabaringiz yetdi!".encode("utf-8"))
klient_sock.close()
server.close()
# === KLIENT ===
import socket

klient = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
klient.connect(("127.0.0.1", 12345))

klient.send("Salom, Server!".encode("utf-8"))

javob = klient.recv(1024).decode("utf-8")
print(f"Server javobi: {javob}")

klient.close()

```

7. Xulosa Jadvali

Metod	Tomon	Vazifasi	Blokirovchimi?
socket()	Ikkalasi	Socket yaratish	✗
bind()	Server	IP:Port biriktirish	✗
listen()	Server	Kutish rejimi	✗
accept()	Server	Ulanish qabul qilish	✓ Kutadi
connect()	Klient	Serverga ulanish	✓ Kutadi
send()	Ikkalasi	Ma'lumot yuborish	✓ Kutadi
recv()	Ikkalasi	Ma'lumot olish	✓ Kutadi
close()	Ikkalasi	Ulanishni yopish	✗

💡 **Eslab qol:** `accept()` va `recv()` — blokirovchi metodlar, shuning uchun real dasturlarda ular **alohida thread** da ishga tushiriladi. Aks holda dasturning butun interfeysi muzlab qoladi!

1. Nima uchun Tarmoq Xatoliklari Maxsus?

Oddiy dasturlashdagi xatolar **deterministik** — ya'ni bir xil sharoitda har doim bir xil natija beradi. Tarmoq xatoliklari esa **nondeterministik** — har safar boshqacha bo'lishi mumkin:

```
Oddiy xato:      2 + "a" → har doim TypeError
Tarmoq xato:     server.connect() → ba'zan ishlaydi,
                                           ba'zan ConnectionRefused,
                                           ba'zan Timeout,
                                           ba'zan BrokenPipe...
```

Sabablar:

- Internet uzilishi
- Server o'chib qolishi
- Firewall bloklashi
- Tarmoq kechikishi (latency)
- Server band bo'lishi
- Paket yo'qolishi

Shuning uchun har bir tarmoq operatsiyasi **himoyalangan** bo'lishi shart.

2. Asosiy Xato Turlari va Sabablari

● ConnectionRefusedError

```
klient.connect(("127.0.0.1", 12345))
# → ConnectionRefusedError
```

Sababi: Server o'sha portda ishlamayapti yoki firewall bloklayapti. OS dan "Bu portga hech kim quloq solmayapti" degan javob keladi. TCP darajasida RST (Reset) paketi qaytariladi.

```
Klient → SYN → Server
Klient ← RST ← Server ← "Yo'q, men bu portda yo'qman"
→ ConnectionRefusedError
```

● socket.timeout

```
klient.settimeout(5.0)
klient.recv(1024)
# → socket.timeout (5 soniya o'tsa)
```

Sababi: Belgilangan vaqt ichida server javob bermadi. Ikki xil timeout mavjud:

```
Connection timeout → serverga ulanishga urinayotganda
Read timeout       → ma'lumot kelishini kutayotganda
```

Timeout o'rnatilmasa — `recv()` **abadiy** kutishi mumkin! Bu dasturni muzlatib qo'yadi.

● BrokenPipeError / ConnectionResetError

```
klient.send(data)
```

```
# → BrokenPipeError yoki ConnectionResetError
```

Sababi: Ulanish o'rnatilgan edi, lekin ma'lumot yuborganda narigi tomon allaqachon ulanishni uzib qo'ygan. Xuddi telefonda gaplashayotganda qo'ng'iroq uzilgan kabi.

BrokenPipeError → Linux/Mac da (SIGPIPE signali)

ConnectionResetError → Windows da (RST paketi)

Ikkisi aslida bir xil holat — platform farqi

● OSError va socket.error

socket.error → OSError ning subklassi

Barcha past darajadagi tarmoq xatolari

Iyerarxiya:

OSError

└─ socket.error

 └─ ConnectionRefusedError

 └─ ConnectionResetError

 └─ ConnectionAbortedError

 └─ TimeoutError (≈ socket.timeout)

3. try-except Strategiyalari

✗ Noto'g'ri usul — Hamma narsani ushlab tashlash:

```
try:
```

```
    klient.connect(("127.0.0.1", 12345))
```

```
    klient.send(data)
```

```
except Exception as e:
```

```
    print("Xato!") # ← nima xato ekanini bilmaymiz
```

Muammo: Qaysi xato ekani noma'lum, har xil xatolar har xil yechim talab qiladi.

✓ To'g'ri usul — Xato turini aniq ajratish:

```
import socket
```

```
def serverga_ulan(host, port, timeout=5.0):
```

```
    klient = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

```
    klient.settimeout(timeout) # ← timeout o'rnatish shart!
```

```
    try:
```

```
        klient.connect((host, port))
```

```
    return klient
```

```

except ConnectionRefusedError:
    # Server o'chiq yoki port yopiq
    print(f"❌ Server {host}:{port} da ishlamayapti.")
    return None

except socket.timeout:
    # Timeout – server javob bermadi
    print(f"⌚ {timeout} soniya ichida server javob bermadi.")
    return None

except OSError as e:
    # Tarmoq yo'q, noto'g'ri manzil va boshqalar
    print(f"🌐 Tarmoq xatosi: {e}")
    return None

```

4. Timeout — Eng Muhim Himoya

Timeout o'rnatilmagan socket xavfli! `recv()` chaqirilganda dastur abadiy kutib qolishi mumkin.

```

klient = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# Ulanish uchun timeout
klient.settimeout(10.0)    # 10 soniya

try:
    klient.connect(("server.com", 80))
except socket.timeout:
    print("Ulanish vaqti o'tdi")
    klient.close()
    return

# Ulanish muvaffaqiyatli – ma'lumot olish uchun
klient.settimeout(30.0)    # 30 soniya (ma'lumot olish uchun ko'proq vaqt)

try:
    javob = klient.recv(4096)
except socket.timeout:
    print("Javob kelishini kutib bo'lmadi")

```

Amaliy qoida:

Ulanish timeout → 3-10 soniya (server tez javob berishi kerak)
 Ma'lumot timeout → 15-60 soniya (katta fayl kelishi mumkin)

5. BrokenPipe — Himoya Usullari

`BrokenPipe` ni oldindan sezib bo'lmaydi — faqat yuborganda bilinadi. Shuning uchun **har bir send()** **himoyalanishi shart:**

```

def xavfsiz_yuborish(sock, data: bytes) -> bool:
    """
    True → muvaffaqiyatli yuborildi

```



```

False → ulanish uzilgan
"""
try:
    sock.sendall(data)
    return True

except BrokenPipeError:
    print("🚰 Ulanish uzildi (BrokenPipe)")
    return False

except ConnectionResetError:
    print("🚰 Server ulanishni uzdi (ConnectionReset)")
    return False

except socket.timeout:
    print("⌚ Yuborish vaqti o'tdi")
    return False

except OSError as e:
    print(f"❌ Yuborish xatosi: {e}")
    return False

```

6. Qayta Urinish (Retry) Strategiyasi

Tarmoq xatoliklari ko'pincha **vaqtinchalik** bo'ladi. Shuning uchun darhol xato deb hisoblamasdan — **qayta urinish** tavsiya etiladi:

```

import time

def qayta_urinib_ulan(host, port, urinishlar=3, kutish=2.0):
    """
    Ulanish muvaffaqiyatsiz bo'lsa, bir necha marta qayta urinadi.
    Har urinish orasida kutish vaqti ortib boradi (exponential backoff).
    """
    for i in range(1, urinishlar + 1):
        print(f"🔄 Urinish {i}/{urinishlar}...")
        try:
            sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
            sock.settimeout(5.0)
            sock.connect((host, port))
            print("✅ Ulanish muvaffaqiyatli!")
            return sock

        except ConnectionRefusedError:
            print(f"❌ Server javob bermadi ({i}-urinish)")

        except socket.timeout:
            print(f"⌚ Timeout ({i}-urinish)")

        except OSError as e:
            print(f"🌐 Tarmoq xatosi: {e}")

    if i < urinishlar:
        vaqt = kutish * i # 2, 4, 6 soniya (exponential backoff)
        print(f"⌚ {vaqt} soniyadan so'ng qayta urinamiz...")

```

```
time.sleep(vaqt)

print("🚫 Barcha urinishlar muvaffaqiyatsiz!")
return None
```

Exponential backoff nima?

1-urinish → darhol
2-urinish → 2 soniya kutish
3-urinish → 4 soniya kutish
4-urinish → 8 soniya kutish

Sabab: Server yuklanganida har soniyada urinish serverga yanada ko'proq yuk tushiradi.
Kutish vaqtini oshirish serverni "nafas olishiga" imkon beradi.

7. Foydalanuvchiga Ma'lumot Berish Strategiyalari

Xato xabarlarini foydalanuvchiga to'g'ri yetkazish muhim. Texnik xatolarni to'g'ridan-to'g'ri ko'rsatish yaxshi emas:

```
# ❌ YOMON – texnik xato ko'rsatish:
except ConnectionRefusedError as e:
    print(f"ConnectionRefusedError: [Errno 111] {e}")
    # Foydalanuvchi buni tushunmaydi!

# ✅ YAXSHI – tushunarli xabar:
except ConnectionRefusedError:
    print("Server hozir mavjud emas. Keyinroq urinib ko'ring.")

# ✅ YAXSHI – logging bilan (texnik ma'lumot saqlanadi):
import logging
logging.basicConfig(filename="xatolar.log", level=logging.ERROR)

except ConnectionRefusedError as e:
    logging.error(f"Ulanish rad etildi: {e}")           # Log ga
    print("Server hozir mavjud emas.")                 # foydalanuvchiga
```

PyQt5 da foydalanuvchiga xabar berish:

```
from PyQt5.QtWidgets import QMessageBox

def xato_korsot(self, xato_turi: str):
    xabarlar = {
        "connection_refused": (
            "Server topilmadi",
            "Server ishlamayapti yoki manzil noto'g'ri.\n"
            "Server ishga tushganini tekshiring."
        ),
        "timeout": (
            "Ulanish vaqti o'tdi",
            "Server uzoq vaqt javob bermadi.\n"
            "Internet aloqangizni tekshiring."
        ),
        "broken_pipe": (
            "Ulanish uzildi",
```

```

        "Server bilan aloqa to'xtatildi.\n"
        "Qayta ulanishga harakat qilinmoqda..."
    ),
}

sarlavha, matn = xabarlar.get(
    xato_turi,
    ("Noma'lum xato", "Kutilmagan xato yuz berdi.")
)
MessageBox.critical(self, sarlavha, matn)

```

8. To'liq Himoyalangan Klient Namunasi

```

import socket
import logging
import time

logging.basicConfig(
    filename="tarmoq.log",
    level=logging.ERROR,
    format="%(asctime)s - %(levelname)s - %(message)s"
)

class HimoyalangaKlient:
    def __init__(self, host, port):
        self.host = host
        self.port = port
        self.sock = None

    def ulan(self) -> bool:
        try:
            self.sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
            self.sock.settimeout(5.0)
            self.sock.connect((self.host, self.port))
            return True

        except ConnectionRefusedError as e:
            logging.error(f"Ulanish rad etildi: {e}")
            print("❌ Server ishlayapti.")
            return False

        except socket.timeout as e:
            logging.error(f"Ulanish timeout: {e}")
            print("⌚ Server javob bermadi.")
            return False

        except OSError as e:
            logging.error(f"Tarmoq xatosi: {e}")
            print(f"🌐 Tarmoq muammosi: {e}")
            return False

    def yuborish(self, xabar: str) -> str | None:
        if not self.sock:
            print("⚠️ Avval ulanish o'rnating!")
            return None

```

```

try:
    self.sock.settimeout(10.0)
    self.sock.sendall(xabar.encode("utf-8"))
    javob = self.sock.recv(4096).decode("utf-8")
    return javob

except BrokenPipeError:
    logging.error("BrokenPipe: ulanish uzildi")
    print("🔊 Ulanish uzildi, qayta ulanish...")
    self.sock = None
    return None

except ConnectionResetError:
    logging.error("ConnectionReset: server uzdi")
    print("🔊 Server ulanishni uzdi.")
    self.sock = None
    return None

except socket.timeout:
    logging.error("Recv timeout")
    print("🕒 Javob kutish vaqti o'tdi.")
    return None

def yopish(self):
    if self.sock:
        try:
            self.sock.close()
        except OSError:
            pass
        finally:
            self.sock = None
            print("🔊 Ulanish yopildi.")

```

9. Xulosa

Xato turi	Sabab	Yechim
ConnectionRefused	Server yo'q/port yopiq	Qayta urinish
socket.timeout	→ Javob kelmadi	→ settimeout() + timeout handling
BrokenPipe	→ Ulanish uzildi	→ sendall() ni try-except ichida
OSError	→ Umumiy tarmoq xato	→ Logga yozish + foydalanuvchiga tushunarli xabar

Asosiy qoidalar:

- ✅ Har doim settimeout() o'rnat
- ✅ Har bir tarmoq operatsiyasi try-except ichida
- ✅ Xato turlarini aniq ajrat
- ✅ Foydalanuvchiga texnik emas, tushunarli xabar ber
- ✅ Texnik ma'lumotni log fayliga yoz
- ✅ Vaqtinchalik xatolar uchun qayta urinish mexanizmi qo'sh

💡 **Eslab qol:** Tarmoqda hech narsa kafolatlanmagan — har doim "Agar bu ulanish uzilsa nima bo'ladi?" deb o'yla!

● SAVOL 10:

KO'P FOYDALANUVCHILI CHAT SERVERNI QURISH UCHUN QANDAY ARXITEKTURA TANLASH KERAK? `THREADING` VA `ASYNCIO` YONDASHUVLARINI SOLISHTIRING, KICHIK LOYIHA UCHUN QAYSI BIRI AFZAL VA NIMA UCHUN?

1. Muammo nima?

Oddiy server faqat **bitta** klientga xizmat qila oladi:

Oddiy server:

```
Klient A → Ulanadi → Server A bilan band
Klient B → Kutadi... kutadi... kutadi...
Klient C → Kutadi... kutadi... kutadi...
```

Ko'p foydalanuvchili chat uchun server **bir vaqtda** ko'plab klientlar bilan ishlashi shart. Buning uchun ikki yondashuv mavjud: **Threading** va **Asyncio**.

2. Threading — Ko'p Oqimli Yondashuv

G'oya: Har bir klient uchun alohida **thread** (oqim) yaratiladi. Har bir thread mustaqil ishlaydi.

```
Server
├─ Thread 1 → Klient A bilan ishlaydi
├─ Thread 2 → Klient B bilan ishlaydi
├─ Thread 3 → Klient C bilan ishlaydi
└─ Thread 4 → Klient D bilan ishlaydi
```

Qanday ishlaydi:

```
accept() → yangi klient keldi
↓
threading.Thread(target=klient_handler) → yangi thread yaratiladi
↓
Thread o'z ichida recv()/send() ni chaqiradi
↓
Thread blokirovchiga kirganda (recv) → OS boshqa threadni ishlatadi
```

Nazariy asos — Blokirovchi I/O: Threading da `recv()` chaqirilganda thread **uxlaydi** — OS uni to'xtatib, boshqa threadni ishga tushiradi. Ma'lumot kelganda OS yana shu threadni uyg'otadi. Bu **OS darajasida kontekst almashtirish** (context switching) deyiladi.

```
import threading
import socket
```

```
klientlar = [] # barcha klientlar ro'yxati
lock = threading.Lock() # ro'yxatga xavfsiz murojaat uchun
```

```

def klient_handler(sock, manzil):
    with lock:
        klientlar.append(sock)

    try:
        while True:
            xabar = sock.recv(1024)
            if not xabar:
                break
            # Barcha klientlarga yuborish
            with lock:
                for k in klientlar:
                    if k != sock:
                        try:
                            k.send(xabar)
                        except:
                            pass
    finally:
        with lock:
            klientlar.remove(sock)
        sock.close()

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(("0.0.0.0", 12345))
server.listen(10)

while True:
    sock, manzil = server.accept()
    t = threading.Thread(target=klient_handler, args=(sock, manzil))
    t.daemon = True # asosiy dastur tugaganda thread ham tugaydi
    t.start()

```

3. Threading ning Muammolari

● GIL (Global Interpreter Lock)

Python da **GIL** mavjud — bir vaqtda faqat bitta thread Python kodini bajara oladi. Bu threading ning samaradorligini cheklaydi.

C++/Java threading:	Thread 1 va Thread 2 haqiqatan parallel
Python threading:	Thread 1 ishlaydi → GIL → Thread 2 ishlaydi (aslida navbat bilan, parallel emas!)

Lekin: Tarmoq operatsiyalarida (`recv` , `send`) GIL **bo'shatiladi** — chunki bu Python kodi emas, OS kodi. Shuning uchun chat server uchun threading hali ham samarali.

● Xotira va tezlik muammosi

100 klient → 100 thread → ~100MB xotira
 1000 klient → 1000 thread → ~1GB xotira ← muammo!

Thread yaratish vaqti: ~1ms
Thread kontekst almashtirish: ~10-100 mikrosekund

● Race Condition — Mushtarak ma'lumotlarga xavf

```
# ❌ XAVFLI – lock ishlatilmagan:
klientlar.append(sock)    # Thread 1
klientlar.remove(sock)    # Thread 2 – bir vaqtda!
# → Ro'yxat buzilishi mumkin

# ✅ XAVFSIZ – lock bilan:
with lock:
    klientlar.append(sock)
```

4. Asyncio — Asinxron Yondashuv

G'oya: Bitta thread ichida ko'plab vazifalarni **navbat bilan** bajarish. Bir vazifa kutayotganda, boshqasi ishlaydi.

Asyncio Event Loop (bitta thread):

```
Klient A recv() kutmoqda →
    Klient B ga xabar yuborilmoqda →
        Klient C recv() kutmoqda →
            Klient A dan xabar keldi! →
                Klient A ni qayta ishlatish
```

Nazariy asos — Non-blocking I/O: Asyncio da `await recv()` chaqirilganda thread **uxlamaydi** — faqat "men hozir band emasman, boshqasi ishlasin" deydi. Bu **kooperativ multitasking** — vazifalar o'zlari qachon "bo'shashlari"ni belgilaydi.

Threading: OS qaror qiladi qaysi thread ishlashini (preemptive)
Asyncio: Kod o'zi qaror qiladi qachon boshqasiga joy berish (cooperative)

```
import asyncio
```

```
klientlar = set()
```

```
async def klient_handler(reader, writer):
    manzil = writer.get_extra_info("peername")
    klientlar.add(writer)

    try:
        while True:
            xabar = await reader.read(1024) # ← await: boshqalar ishlayversin
            if not xabar:
                break

            # Barcha klientlarga yuborish
            for k in klientlar:
                if k != writer:
                    k.write(xabar)
                    await k.drain() # ← buffer to'ldirilishi uchun
```

```

    finally:
        klientlar.discard(writer)
        writer.close()

async def main():
    server = await asyncio.start_server(
        klient_handler, "0.0.0.0", 12345
    )
    async with server:
        await server.serve_forever()

asyncio.run(main())

```

5. Asyncio ning Afzalligi va Kamchiligi

Afzalligi — Resurslar:

1000 klient, Threading: 1000 thread → ~1GB xotira
 1000 klient, Asyncio: 1 thread → ~50MB xotira

Asyncio 20x kam xotira sarflaydi!

Afzalligi — Tezlik:

Thread yaratish: ~1ms
 Asyncio task yaratish: ~1 mikrosekund (1000x tez!)

Kamchiligi — Murakkablik:

```

# Threading: oddiy, tushunarli
def handler():
    data = sock.recv(1024) # to'g'ridan-to'g'ri

# Asyncio: async/await tushunish kerak
async def handler():
    data = await reader.read(1024) # nima uchun await?

```

Kamchiligi — Blokirovchi kod xavfi:

```

# ❌ ASYNCIO DA XAVFLI – blokirovchi funksiya:
async def handler():
    import time
    time.sleep(5) # ← BUTUN serverni muzlatadi!
    # Asyncio bitta thread – bu yerda hamma kutib qoladi

# ✅ TO'G'RI:
async def handler():
    await asyncio.sleep(5) # ← faqat bu task kutadi

```

6. Threading vs Asyncio — To'liq Taqqoslash

Xususiyat	Threading	Asyncio
Tushunarlilik	✓ Oson	✗ Murakkab
Xotira sarfi	✗ Ko'p	✓ Kam
100 klient	✓ Yaxshi	✓ Yaxshi
10,000 klient	✗ Qiyin	✓ Ideal
Race condition	✗ Xavf bor	✓ Yo'q
GIL muammosi	⚠ Bor	✓ Yo'q
Debug qilish	✓ Oson	✗ Qiyin
Blokirovchi kutubxona	✓ Ishlaydi	✗ Ishlamaydi
O'rganish vaqti	✓ Kam	✗ Ko'p
Kichik loyiha	✓ Ideal	⚠ Ortiqcha

7. Kichik Loyiha Uchun Qaysi Biri Afzal?

Kichik loyiha uchun — Threading tavsiya etiladi. Sabablari:

✓ Sabab 1 — Tushunarlilik

Threading → oddiy funksiyalar, tanish kod

Asyncio → async/await, event loop, coroutine – yangi paradigma

Kichik loyihada vaqt tejash muhim. Threading ni tushunish uchun 1 soat, Asyncio uchun 1 hafta kerak bo'lishi mumkin.

✓ Sabab 2 — Kichik loyihada foydalanuvchi soni kam

0-50 klient → Threading mukammal ishlaydi

50-500 klient → Threading hali yaxshi

500+ klient → Asyncio afzal bo'la boshlaydi

Yakuniy nazorat yoki kichik chat dasturi uchun 50 klientdan oshishi dargumon.

✓ Sabab 3 — Debug qilish osonligi

Threading xatosi: oddiy traceback → tuzatish oson

Asyncio xatosi: coroutine stack → tushunish qiyin

✓ Sabab 4 — Kutubxonalar bilan moslik

PyQt5 + Threading → muammosiz

self.thread = threading.Thread(target=server_handler)

PyQt5 + Asyncio → qo'shimcha ishlar kerak

asyncio event loop va Qt event loop ni birlashtirish

qiyin va alohida kutubxona (qasync) talab qiladi

8. Qachon Asyncio tanlash kerak?

Kichik loyiha	→ Threading	✓
O'rta loyiha	→ Threading	✓ (500 klientgacha)
Katta loyiha	→ Asyncio	✓
Real vaqt chat	→ Asyncio	✓ (WebSocket bilan)
PyQt5 bilan	→ Threading	✓ (osonroq)
Production server	→ Asyncio	✓ (yoki aralash)

9. Xulosa

Threading:

- Har klient uchun alohida thread
- OS boshqaradi (preemptive)
- Oson, tushunarli, PyQt5 bilan mos
- Xotira ko'p, race condition xavfi

Asyncio:

- Bitta thread, ko'p vazifa
- Kod o'zi boshqaradi (cooperative)
- Tez, kam xotira, race condition yo'q
- Murakkab, blokirovchi kod xavfli

Kichik loyiha uchun javob:

- ✓ THREADING – oson, tez o'rganiladi, kichik miqyos uchun to'liq yetarli, PyQt5 bilan muammosiz ishlaydi

💡 **Eslab qol:** "Eng yaxshi arxitektura" — loyihangiz talabini qondirgan, eng tez yozib, debug qila oladigan arxitektura. Kichik loyihada Asyncio ishlatish — sport mashinasiga maktabga borish kabi: texnik jihatdan mumkin, lekin amaliy jihatdan ortiqcha!